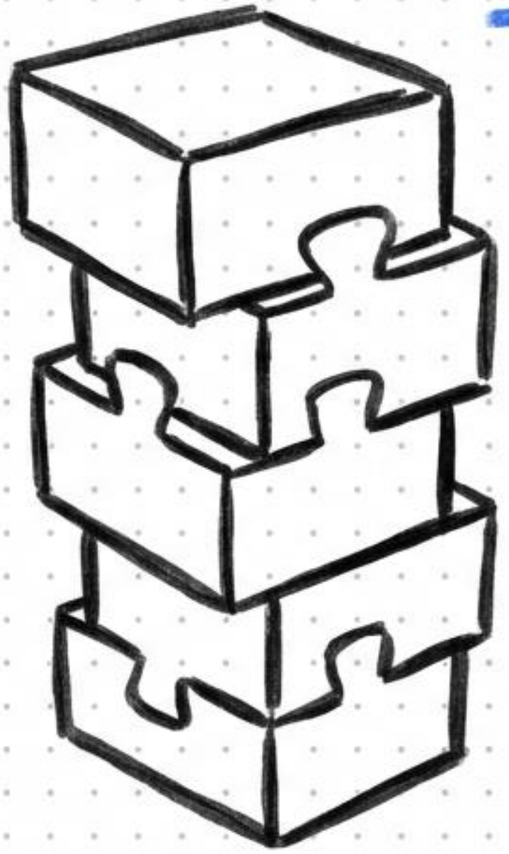
The 5-Layer Customization Model



Architecting Your AI Teammate

A progressive guide to customizing GitHub Copilot from instructions to the SDK.

The "Mega-Prompt" Anti-Pattern

- Stuffing workflows into a single prompt wastes context tokens.
- Mixes personality with procedural runbooks.
- Leads to unpredictable LLM deviation.

Instructions are standards, not runbooks.
We need a system, not just a prompt.

The Anatomy of an AI Teammate

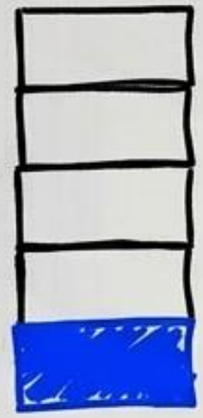
Copilot SDK → *The Office (Where they work)*

Hooks → *Security Checkpoints (How we stay safe)*

Custom Agents → *Dedicated Roles (Who does what)*

Agent Skills → *Specialist Manuals (How to do tasks)*

Instructions → *Company Handbook (Who you are)*



Layer 1: Instructions

The always-on foundation of your AI's personality and standards.

`.github/copilot-instructions.md`

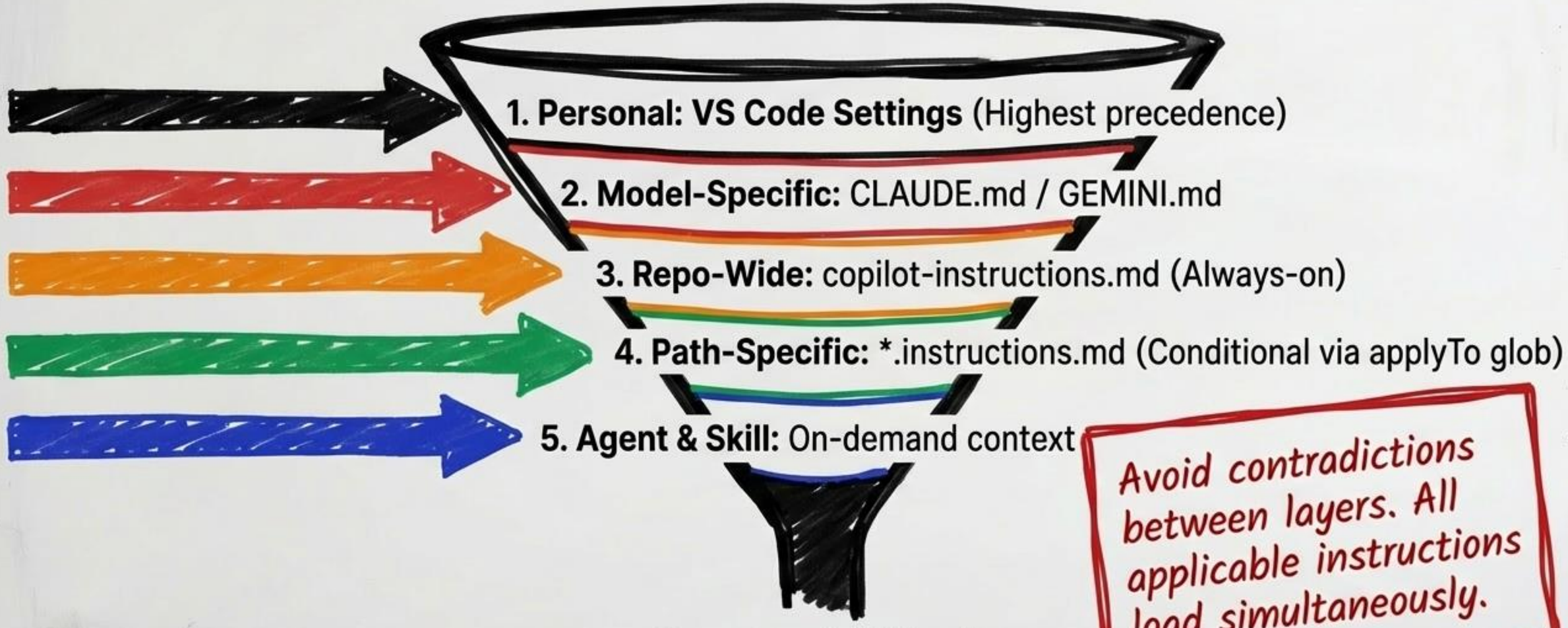
```
- Use TypeScript strict mode
- Prefer pnpm over npm
- Never commit secrets
```

Scope: Global coding standards.

Rule of Thumb: 5–15 core rules max (200–500 tokens).

Anti-Pattern: Do NOT include workflow orchestration here.

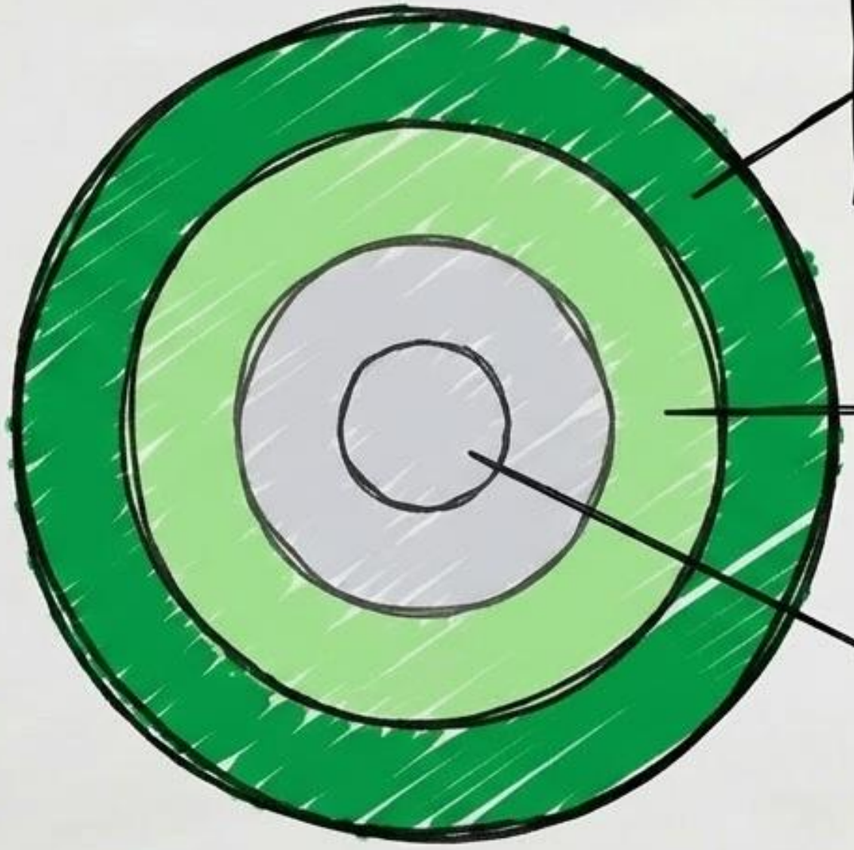
The Precedence Funnel



Avoid contradictions between layers. All applicable instructions load simultaneously.

Layer 2: Agent Skills

On-demand specialist training to conserve tokens.



Tier 1: Metadata
Always-on. ~50-100 tokens. Driven entirely by the YAML description.

Tier 2: Body (SKILL.md)
Loaded on description match. < 500 tokens.

Tier 3: References
Loaded via explicit reference. < 1000 tokens per file.

Structuring a Skill

```
.github/skills/webapp-testing/
│
│
│
├── SKILL.md (Required: The primary instructions)
├── scripts/helper.py (Executable logic)
└── references/api_reference.md (On-demand docs)
```

The **description** field in the YAML frontmatter is the **SOLE** criterion Copilot uses to trigger the skill. **Be precise!**

Layer 3: Custom Agents (.agent.md)

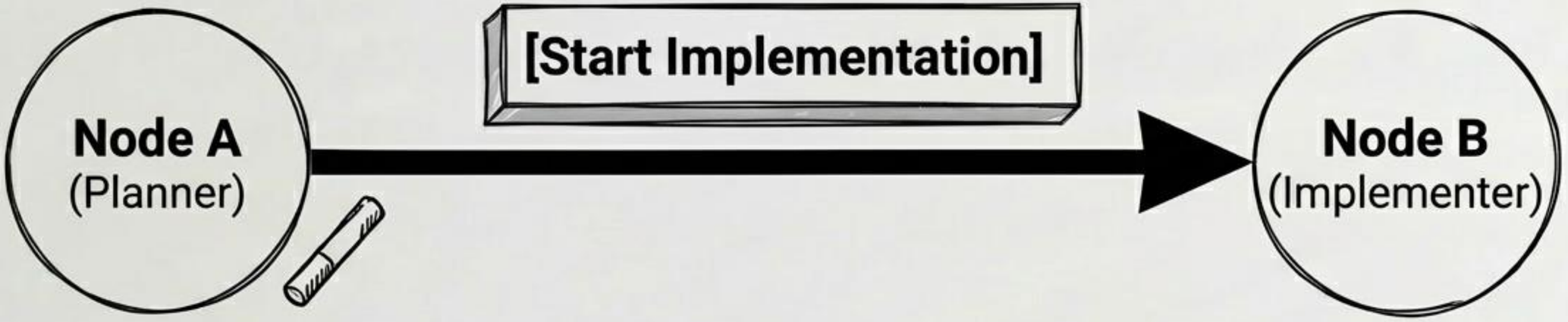
Defining specialized roles and how they collaborate.

Mode 1: Handoffs (User-Guided)

Mode 2: Sub-Agents (Programmatic)

Which collaboration mode do you need?

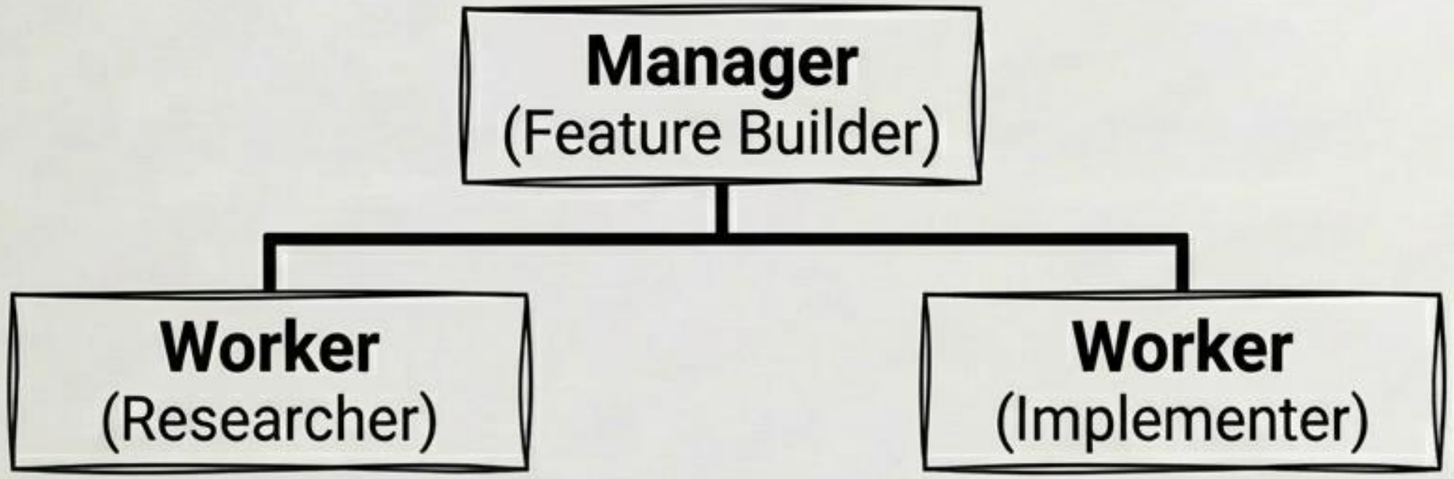
The Handoff: A Relay Race



Mechanics:
- Trigger: User clicks a UI button.
- Control: Human-in-the-loop validation.
- Context: Shared conversation history carries over.

Best For:
Multi-stage pipelines requiring human review (e.g., Plan -> Review -> Implement).

Sub-Agents: The Org Chart



Mechanics:
- Trigger: Parent agent invokes programmatically.
- Control: Agent-autonomous delegation.
- Context: Clean, isolated context for the sub-agent.

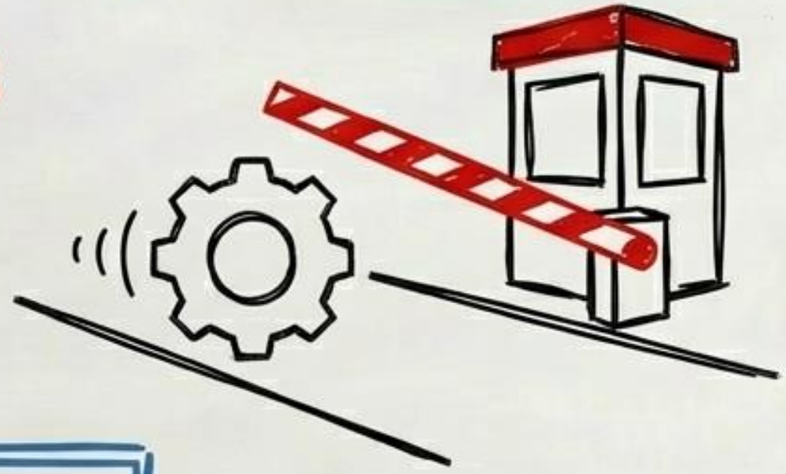
Constraint:
Single-level depth only. Sub-agents cannot invoke other sub-agents.

Orchestration Matrix: Handoff vs. Sub-Agent

	Handoff	Sub-Agent
User Experience	Visible (User sees agent switch)	Transparent (User sees only final result)
Context	Shared history	Isolated context
Depth Limit	Unlimited (back-and-forth)	Single level
Best Suited For	Sequential human-gated pipelines	Automated parallel tasks / review loops

Layer 4: Lifecycle Hooks

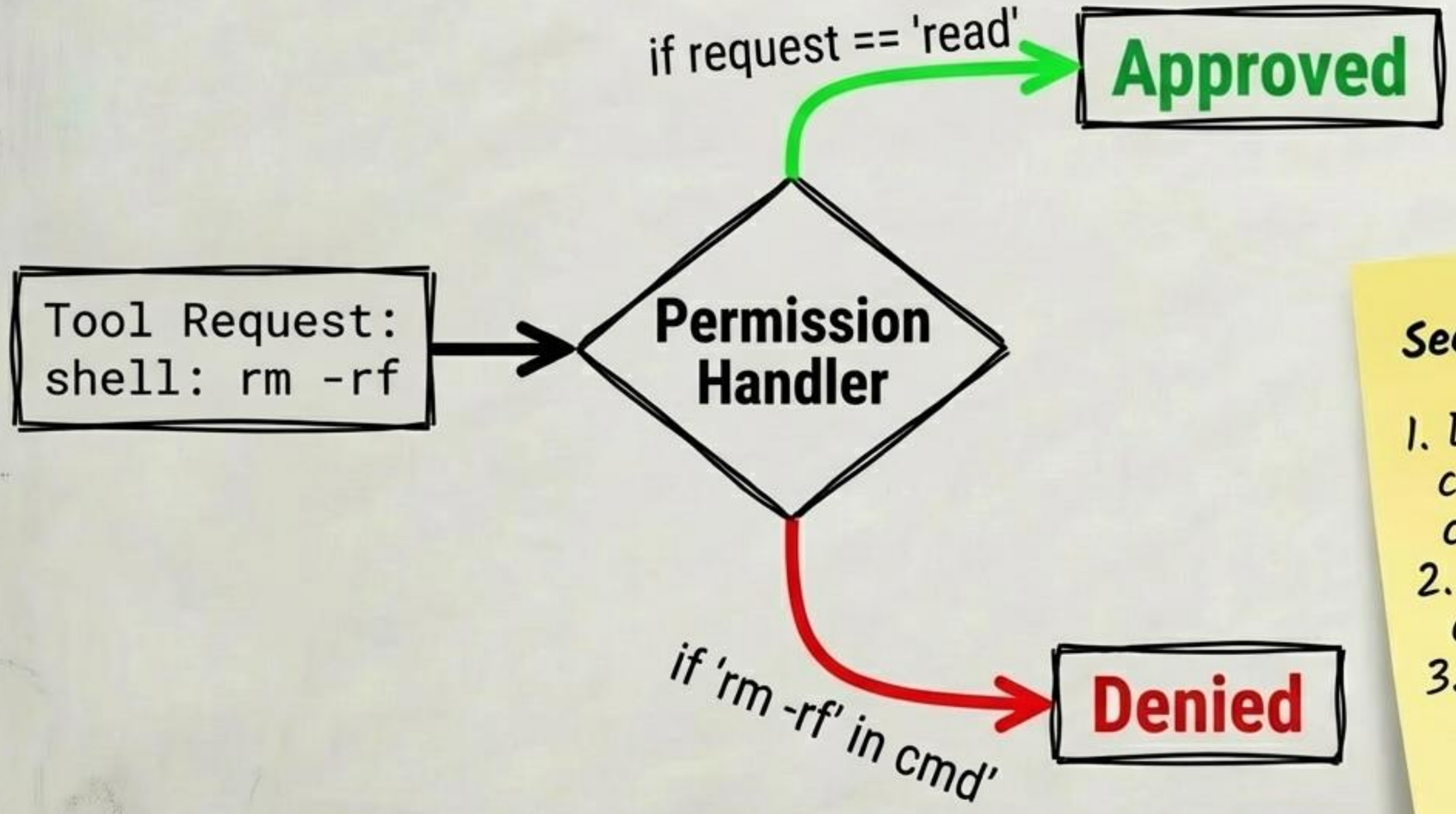
Deterministic code execution outside the LLM.



Key Insight: Hooks are scripts, not prompts. They consume ZERO tokens.

- **Trigger Types:**
  - **PreToolUse**: Permission checks, blocking operations.
  - **PostToolUse**: Linting, logging.
  - **SessionStart / SessionEnd**: Initialization and telemetry.

The Permission Handler



Security Principles:

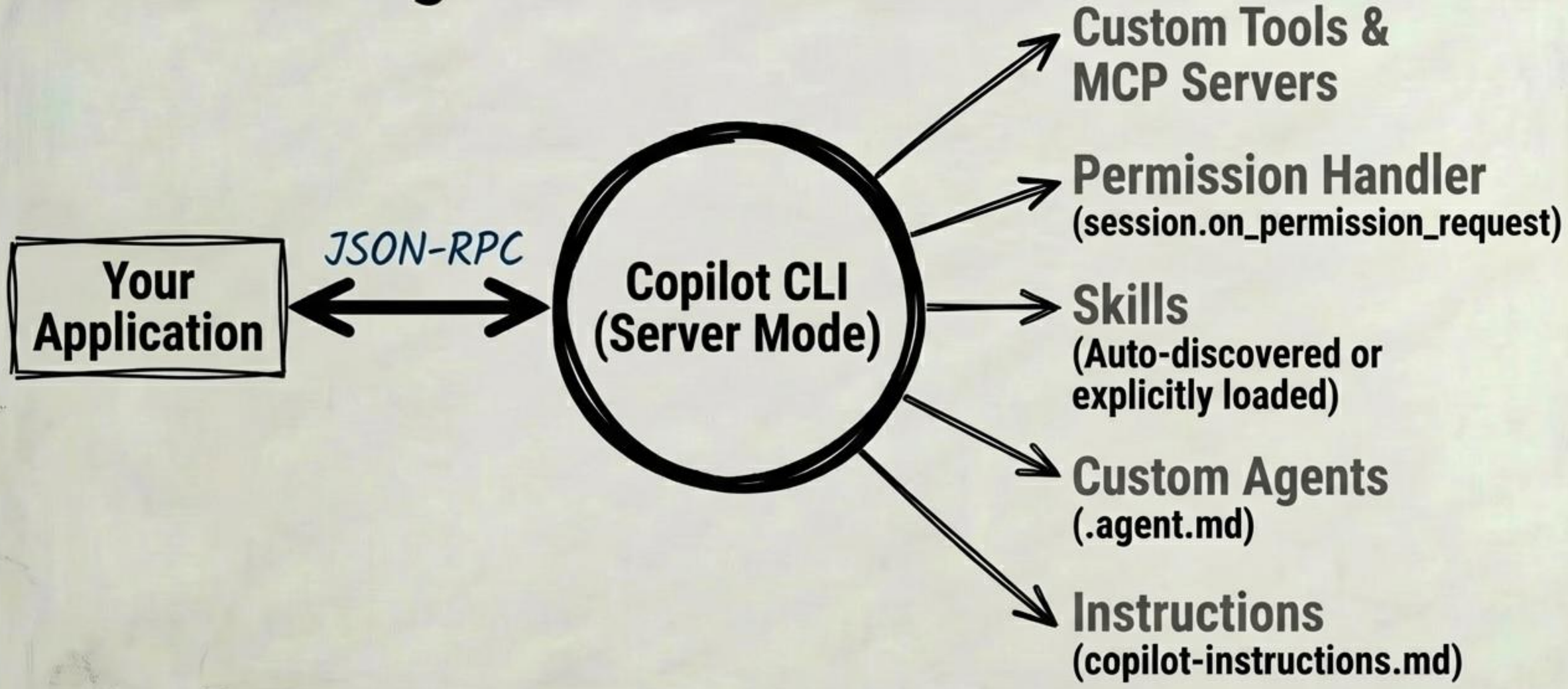
1. Default Deny: Agents cannot execute shell/fs commands by default.
2. Least Privilege: Grant only necessary tools.
3. Container Isolation: Run shell agents in Dev Containers.

Layer 5: The Copilot SDK

Programmatic remote control of the CLI server runtime.

- Wraps the Copilot CLI via JSON-RPC.
- Available in Node.js, Python, Go, and .NET.
- Assembles Instructions, Skills, Agents, and Hooks into custom agentic apps (e.g., Automated PR Reviewers).

The SDK Integration Architecture



Where Does the Workflow Belong?

Is this guidance needed in nearly EVERY interaction?

YES → Put it in copilot-instructions.md.

NO ↓

Is it a task-specific detailed procedure?

YES → Put it in a SKILL.md.

NO ↓

Does it involve inter-agent collaboration?

Human review needed? → .agent.md + Handoff.

Fully automated? → .agent.md + Sub-agent.

The Token Economy Spectrum

0 Tokens	Low / Conditional	Moderate	High Cost
Hooks (Code-level execution)	**Skills** (On-demand, <500 tokens), **Path-Specific Instructions**	**Custom Agent Prompts** (Loaded when selected)	**Repo-Wide Instructions** (Always-on, consumes tokens every invocation)

Strategic Principle: Never always-on what can be loaded on-demand. Never use instructions when hooks suffice.

The Complete Blueprint

```
your-project/
├── .github/
│   .copilot-instructions.md
├── .github/agents/
│   └── planner.agent.md
├── .github/skills/
│   └── db-migration/SKILL.md
├── .vscode/
│   .vscode/settings.json (hooks)
└── src/
    src/analyze_prs.py (SDK script)
```

Layer 1: Global Standards

Layer 3: Roles & Handoffs

Layer 2: On-Demand Workflows

Layer 4: Security

Layer 5: Programmatic Assembly

Building True Agentic Applications

Instructions provide foundational context. **Skills** provide on-demand knowledge. **Custom Agents** divide and conquer. **Hooks** enforce safety safety rules. **The SDK brings it all to life.**

Stop writing mega-prompts. Start engineering architectures.